

## MODULE 8

# Java Project Structure and Modularization

Organize Java applications using a well-defined project structure and modular design for maintainability, scalability, and reuse.













# MODULE 8 OVERVIEW

*A well-structured Java project is the foundation of a successful, maintainable enterprise application.*

- Large enterprise applications may contain thousands of classes — without proper organization, code becomes difficult to maintain, teams conflict during development, and testing becomes difficult.
- This module covers the standard Java/Maven project layout, layered architecture, package organization by responsibility, and how enterprise teams structure real Spring Boot applications.

## DURATION & LAB



### 2 Hours Theory

Core concepts and architecture patterns.



### 45 Min or 3–4 Hr Lab

Timed path (starter) or full extended path.



### Business Scenario

500+ developers, one banking platform.

**KEY TAKEAWAY** Structure is not just about folders — it's about organizing code by feature and responsibility.

# WHY PROJECT STRUCTURE MATTERS

*Today's structure means less complexity tomorrow — it improves code quality, productivity, and long-term maintainability.*

## UNORGANIZED PROJECT

- Files scattered everywhere
- Hard to find and understand code
- High coupling and tight dependencies
- Difficult to test and debug
- Hard to onboard new developers
- Changes are risky and time-consuming

VS

## WELL-STRUCTURED PROJECT

- Easy to navigate and understand
- Clear separation of concerns
- Reduced coupling, higher cohesion
- Easier testing and debugging
- Faster onboarding for new team members
- Safer changes with lower risk

**KEY TAKEAWAY** A strong project structure is the backbone of a maintainable, scalable, and successful Java application.

# LEARNING OBJECTIVES

*By the end of this module, you will organize Java projects effectively and apply modularization principles.*

- 1 Understand the standard Java project layout and its components.
- 2 Organize code using packages and follow naming conventions.
- 3 Apply modularization principles to achieve high cohesion and low coupling.
- 4 Structure code into layers and modular components.
- 5 Manage dependencies using build tools (Maven/Gradle) and configuration files.
- 6 Separate concerns using well-defined modules and packages.
- 7 Improve code reusability through modular design.
- 8 Enhance maintainability and scalability using a structured project.
- 9 Follow best practices for project structure and modularization.
- 10 Build a scalable Java application using a modular project structure.

## WHAT YOU WILL GAIN



**Better Code Organization**



**Easier Navigation**



**Improved Collaboration**



**Faster Onboarding**



**Higher Code Quality**

**KEY TAKEAWAY** Success outcome: design and structure Java applications that are well-organized, modular, and easy to extend.

# ENTERPRISE SCENARIO: LARGE-SCALE BANKING APPLICATION

*GlobalBank must support millions of customers, complex regulations, and constant change — with agility, reliability, and maintainability.*

**Millions**

Customers Across Web, Mobile  
& Branch

**1000s**

Daily Transactions

**100s**

Business Capabilities &  
Workflows

**Strict**

Security, Audit & Regulatory  
Needs

## SCENARIO OVERVIEW

- GlobalBank is a multinational bank with millions of customers and thousands of daily transactions.
- The application must support retail banking, loans, cards, payments, and compliance across multiple regions — with high availability, performance, and scalability.

## BUSINESS DOMAINS



**Retail Banking**



**Loans**



**Cards & Payments**



**Compliance**

**KEY TAKEAWAY** A clear project structure and modular design turn complexity into manageable pieces.



# LARGE SCALE BANKING APPLICATION

## SAMPLE MODULAR STRUCTURE



| MODULES OVERVIEW |                        |                                                          |
|------------------|------------------------|----------------------------------------------------------|
|                  | Customer Management    | Handles customer onboarding, profiles, KYC and AML.      |
|                  | Account Management     | Manages all account related operations and lifecycle.    |
|                  | Transaction Processing | Processes all types of transactions and business rules.  |
|                  | Loan Management        | End-to-end loan lifecycle management.                    |
|                  | Payment Gateway        | Integrates and processes digital payments securely.      |
|                  | Cards Management       | Manages cards, limits and related transactions.          |
|                  | Investments & Wealth   | Handles investment products and wealth management.       |
|                  | Notification           | Sends alerts, notifications and communication.           |
|                  | Fraud Detection        | Monitors and detects fraudulent activities in real-time. |
|                  | Reporting & Analytics  | Generates reports, dashboards and business insights.     |

| DESIGN PRINCIPLES                                                                                                                                                                                                            |  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <ul style="list-style-type: none"><li>Modular &amp; loosely coupled architecture</li><li>High cohesion within modules</li><li>Independent deployable services (optional)</li><li>Scalable, secure and maintainable</li></ul> |  |



This modular structure supports scalability, maintainability, security and enables teams to work independently.

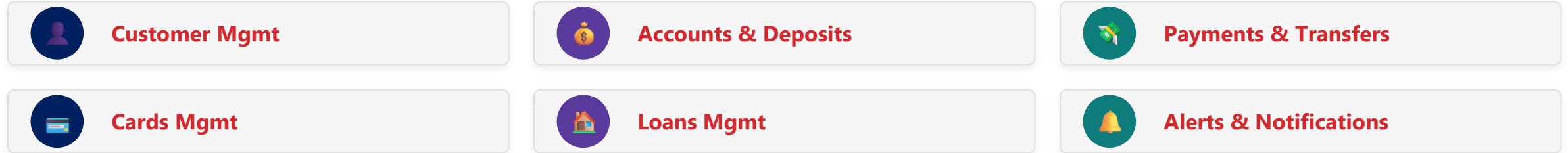
# SAMPLE MODULAR STRUCTURE

*GlobalBank organizes its codebase into channel, business-capability, shared, and foundation modules.*

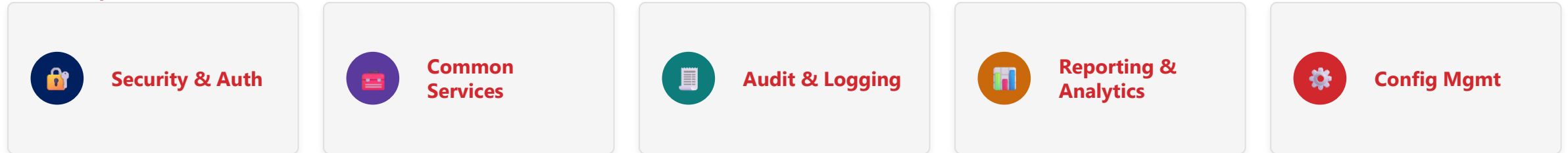
## CHANNELS



## BUSINESS CAPABILITY MODULES



## SHARED / PLATFORM MODULES

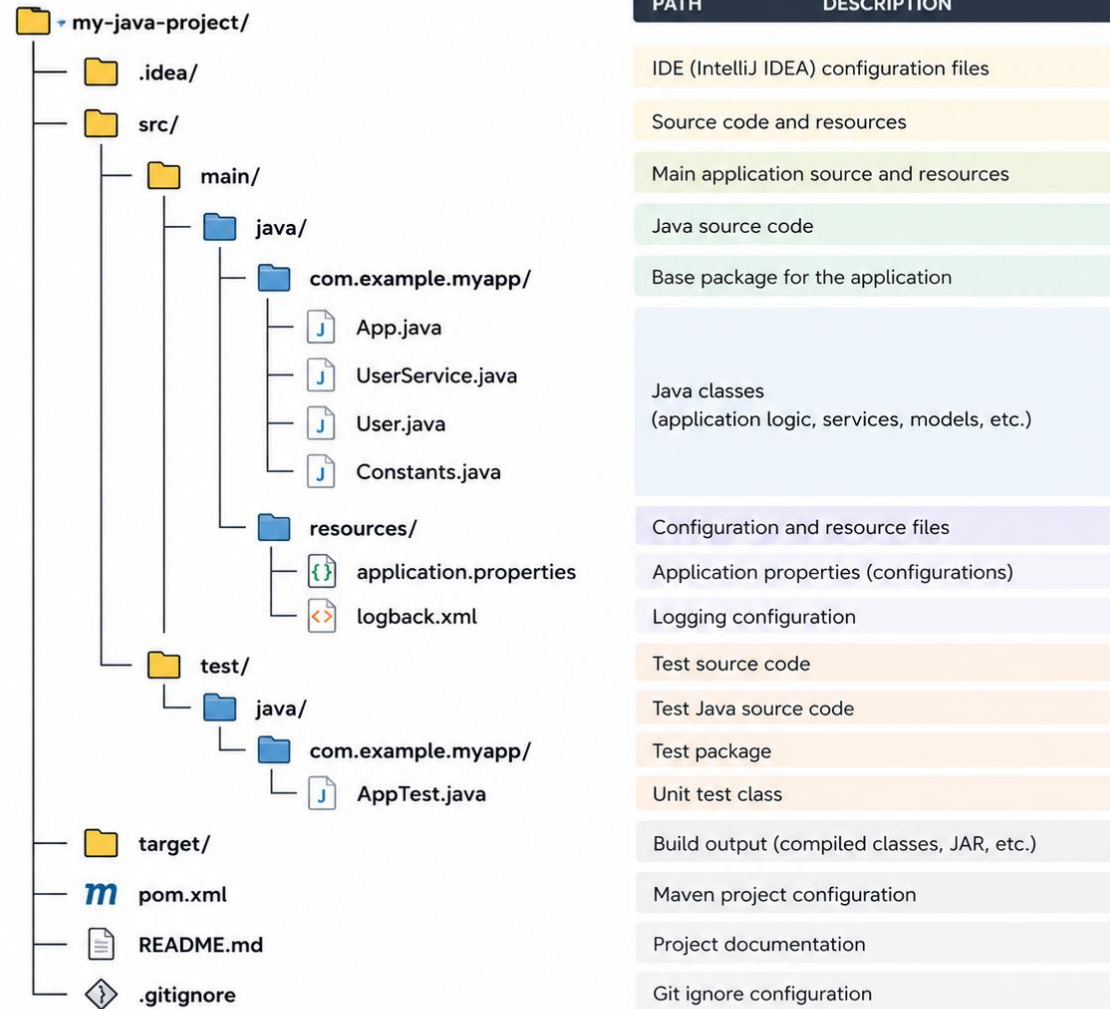


## FOUNDATION



**BUSINESS IMPACT** Cuts technical debt and lets new teams or products (e.g., Digital Wallet) onboard with minimal impact.

# JAVA PROJECT STRUCTURE



## NOTES

- This is a standard Maven-based Java project structure.
- Other build tools (Gradle, Ant) may have slight differences.



# PURPOSE OF EACH DIRECTORY

*Every top-level entry in the standard layout has one clear job.*

| DIRECTORY / FILE       | PURPOSE                                                |
|------------------------|--------------------------------------------------------|
| src/main/java          | Main application Java source files (package structure) |
| src/main/resources     | Configuration files, properties, static resources      |
| src/test/java          | Unit and integration test source code (mirrors main)   |
| pom.xml / build.gradle | Manages dependencies, plugins, and build configuration |
| README.md              | Project overview, setup, and usage instructions        |

**KEY TAKEAWAY** Follows a convention widely used in industry — promotes consistency across projects and teams.

# STANDARD JAVA / MAVEN PROJECT STRUCTURE

*Maven enforces a standard project layout — convention over configuration — that makes builds predictable.*

```
my-maven-project/  
  src/main/java/           // application source code  
  src/main/resources/      // config, static files  
  src/test/java/           // unit + integration tests  
  src/test/resources/      // test resource files  
  pom.xml                  // POM: dependencies, plugins, build  
  target/                  // build output (generated)  
  .mvn/, mvnw, mvnw.cmd    // Maven Wrapper (optional)  
  README.md
```

Example base package (src/main/java/com/example/app/): App.java, service/, repository/, model/, util/

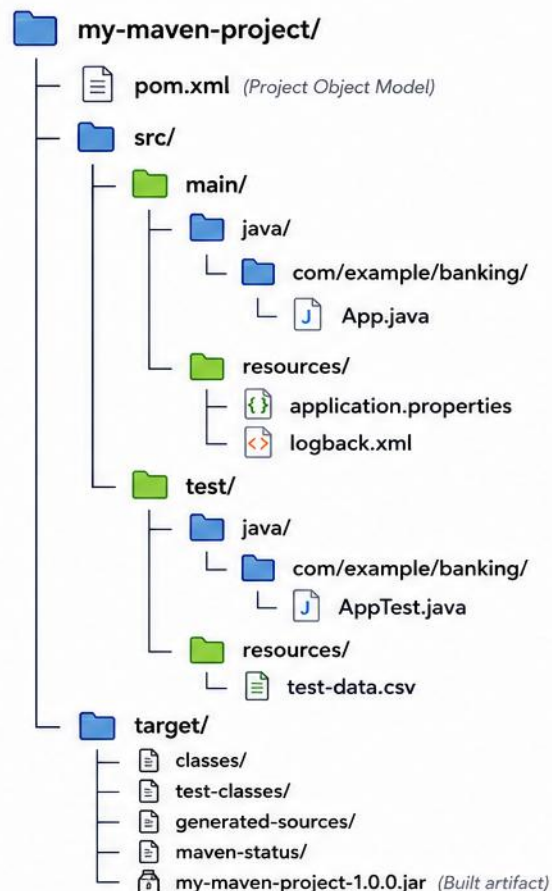
**KEY TAKEAWAY** Maven uses convention over configuration. Lifecycle: validate → compile → test → package → verify → install → deploy.



# MAVEN DIRECTORY LAYOUT



## Standard Project Structure



| DIRECTORY / FILE          | DESCRIPTION                                                                                                                                |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>pom.xml</b>            | Project Object Model.<br>Contains project configuration, dependencies, plugins, build settings.                                            |
| <b>src/main/java</b>      | Contains main Java source code.<br>Follows package structure.                                                                              |
| <b>src/main/resources</b> | Contains resource files used by the application<br>(properties, XML, configuration, etc.).                                                 |
| <b>src/test/java</b>      | Contains unit and integration test source code.<br>Mirrors the main package structure.                                                     |
| <b>src/test/resources</b> | Contains resources for tests<br>(test data, properties, config files).                                                                     |
| <b>target</b>             | Build output directory.<br>Generated during build process.<br>Contains compiled classes, test results, reports,<br>and packaged artifacts. |



### Key Points

- Maven enforces a standard project structure.
- Keeps source code, resources, and tests organized.
- Build outputs are generated in the **target/** directory.



### Benefits

- Convention over configuration
- Easy to maintain and scale
- Works seamlessly with Maven tools and IDEs (IntelliJ, Eclipse, VS Code)



# A MINIMAL POM.XML EXAMPLE

*The Project Object Model (POM) declares the project's coordinates, Java version, and how it builds.*

```
<project>
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example.banking</groupId>
  <artifactId>customer-service</artifactId>
  <version>1.0.0</version>
  <packaging>jar</packaging>

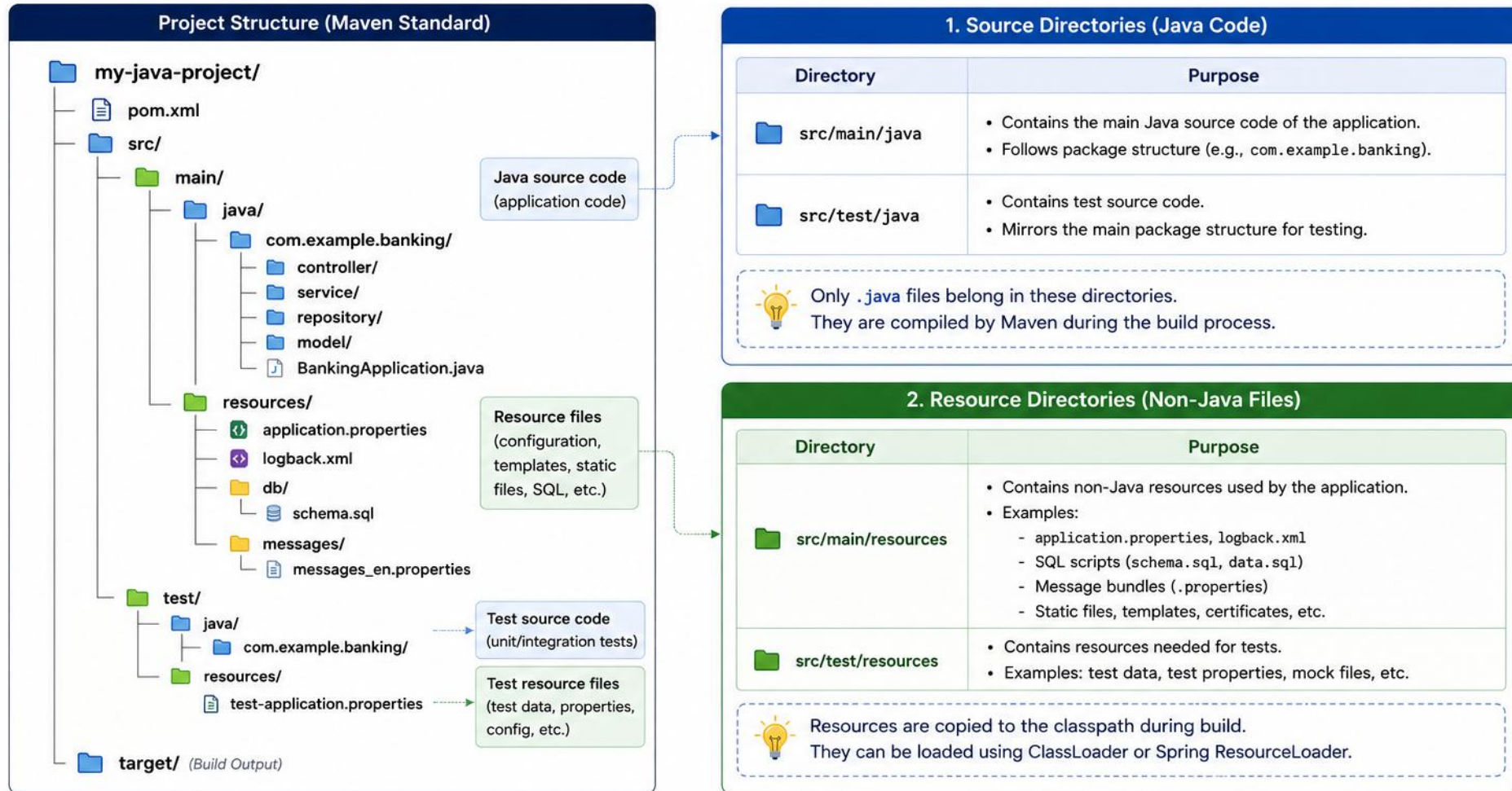
  <properties>
    <maven.compiler.release>21</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <!-- application dependencies go here -->
  </dependencies>
</project>
```

**KEY TAKEAWAY** pom.xml declares what the project IS (groupId/artifactId/version) and HOW it builds (Java release, dependencies) — Maven reads this file for every command.

# Java Source and Resource Directories

Organizing Code and Resources in a Maven Project



## Key Takeaway

Keep all Java code in the **source directories** (src/main/java, src/test/java) and all non-Java files in the **resource directories** (src/main/resources, src/test/resources).



## Benefit:

This structure ensures clean organization, proper build, and easy maintenance.

# TEST DIRECTORY STRUCTURE

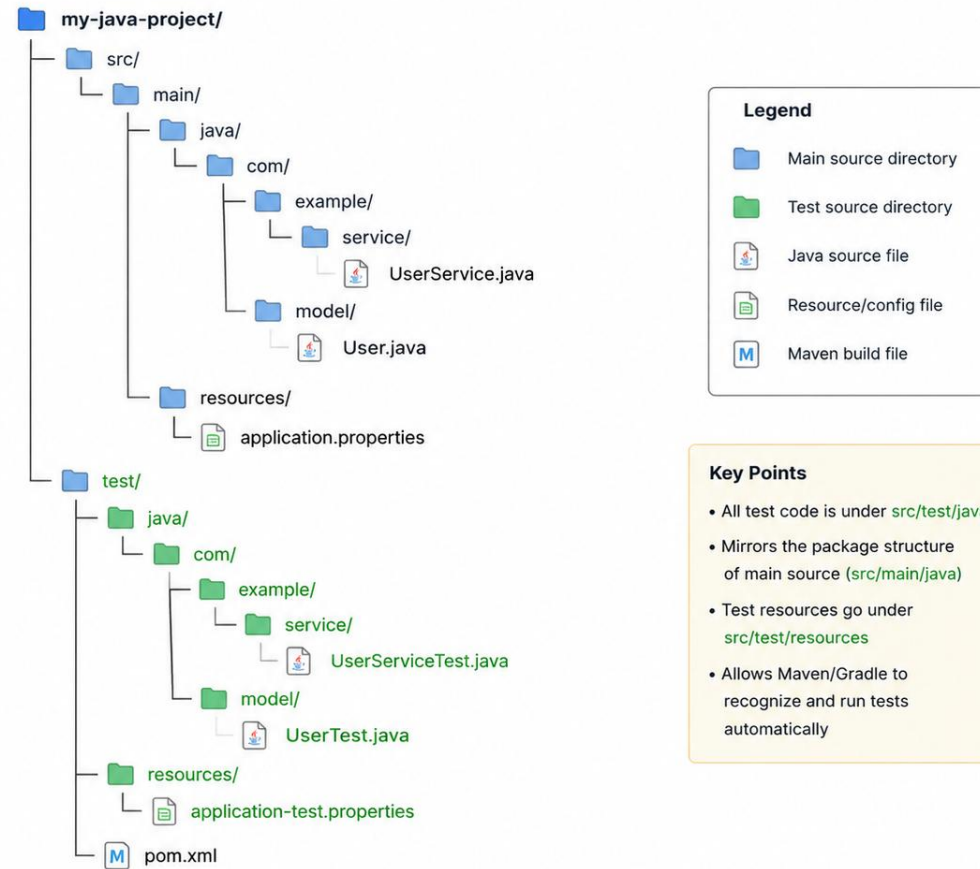
*Test code lives under src/test/java, mirroring the same package structure as main code.*

```
my-maven-project/src/  
  main/java/, main/resources/  
  test/java/com/example/banking/  
    AccountServiceTest.java, AccountRepositoryTest.java  
  test/resources/application-test.properties
```

| TEST TYPE         | LOCATION           | EXAMPLE                     | PURPOSE                                      |
|-------------------|--------------------|-----------------------------|----------------------------------------------|
| Unit Tests        | src/test/java      | AccountServiceTest.java     | Test individual methods/classes in isolation |
| Integration Tests | src/test/java      | AccountRepositoryTest.java  | Test interaction between components/layers   |
| End-to-End Tests  | src/test/java      | MoneyTransferE2ETest.java   | Test the complete business workflow          |
| Test Resources    | src/test/resources | application-test.properties | Provide test data, configs, and utilities    |

**KEY TAKEAWAY** A well-organized test structure makes tests easy to find, run, and maintain. Keep tests fast, independent, and repeatable.

## Java Project – Test Directory Structure



**Example Test Class: UserServiceTest.java**

```
package com.example.service;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class UserServiceTest {
    @Test
    void shouldCreateUser() {
        UserService service = new UserService();
        assertNotNull(service.createUser("John"));
    }
}
```



# TRY IT YOURSELF — EXERCISE 1: READ A MAVEN LAYOUT

*Reinforces: the Maven directory layout — src/main, src/test, docs, target — you just saw.*

| STEP        | WHAT TO DO                                                                            |
|-------------|---------------------------------------------------------------------------------------|
| Goal        | Explain where production code, tests, config, docs, and generated files belong        |
| File        | maven-layout-notes.md (in examples/module-08-exercises/)                              |
| Classify    | Six real files -> src/main/java, src/test/java, src/main/resources, docs/, or target/ |
| Explain     | target/ is generated by Maven — delete and rebuild anytime, so it is .gitignored      |
| Deliverable | Six files correctly classified, target/ explained, no secrets committed in resources  |
| Reference   | exercises/exercise-01-maven-layout.md                                                 |

```
$ tree customer-management-platform
customer-management-platform/
  pom.xml, docs/CODING-STANDARDS.md    // build id + team docs
  src/main/java/, src/test/java/       // production vs. test source
  target/                             // generated by Maven – never commit, always .gitignored
```

**TRY IT NOW** Complete Exercise 1 before continuing — see exercises/exercise-01-maven-layout.md.

# CHOOSING PACKAGE-BY-LAYER VS PACKAGE-BY-FEATURE

*Both are valid choices for different contexts — pick based on project size, team structure, and domain complexity.*

## 1. LAYER (ARCHITECTURAL) APPROACH

- Group by technical responsibility: controller, service, repository, domain.
- Groups related access together.
- Simple and familiar for smaller applications.
- Best for smaller applications and simple domains.

VS

## 2. FEATURE (DOMAIN-DRIVEN) APPROACH

- Group by business feature: customer, account, transaction.
- Each feature package has its own controller, service, repository, model, dto.
- Keeps packages cohesive and focused per feature.
- Best for larger systems, multiple teams, and complex domains.

## 3. HYBRID APPROACH

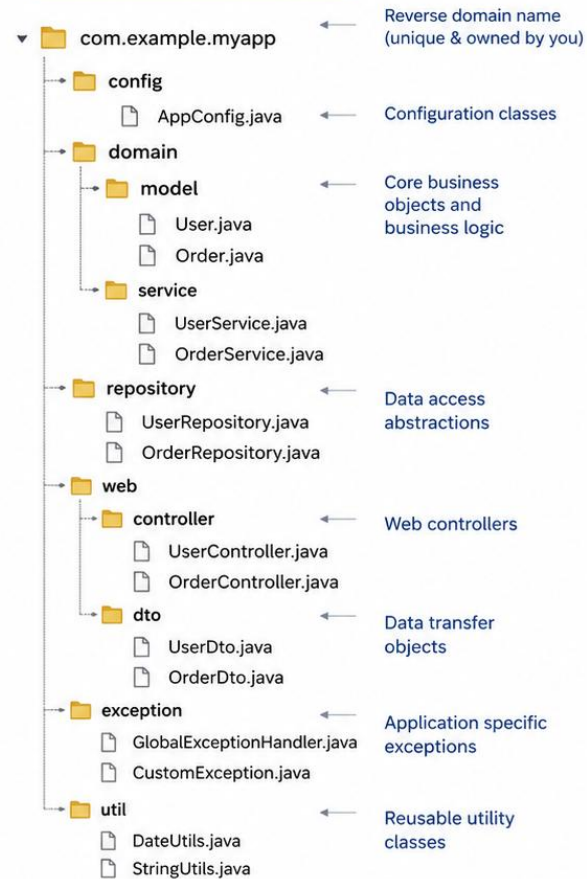
- A feature package containing its own controller/service/repository sub-packages — combines both approaches.
- Best practices: consistent, lowercase, reverse-domain package naming; avoid cyclic dependencies between packages.

**KEY TAKEAWAY** Layer, feature, and hybrid are all valid — choose based on project size, team structure, and domain complexity. Whichever you pick, keep package naming consistent and avoid cyclic dependencies.

# Java Package Organization Best Practices

Well-structured packages improve maintainability, scalability and code clarity.

## EXAMPLE: PACKAGE STRUCTURE



Keep packages focused and cohesive.  
High cohesion, low coupling.

## BEST PRACTICES



### 1. Use Reverse Domain Name

Start with your domain name in reverse order to ensure uniqueness (e.g., com.example.app).



### 2. Organize by Feature / Layer

Group related classes by feature or layer (e.g., controller, service, repository, model).



### 3. High Cohesion, Low Coupling

Keep related classes together.  
Avoid putting unrelated classes in the same package.



### 4. Encapsulate Implementation

Keep internal classes and utilities in appropriate packages. Avoid exposing implementation details.



### 5. Keep It Simple and Flat

Avoid overly deep package nesting.  
2-4 levels is usually enough.



### 6. Use Clear and Meaningful Names

Package names should be lowercase, concise, and meaningful. Avoid abbreviations unless well-known.



### 7. Consistent Across the Project

Follow the same structure and naming conventions throughout the project.



### 8. Put Utilities in the Right Place

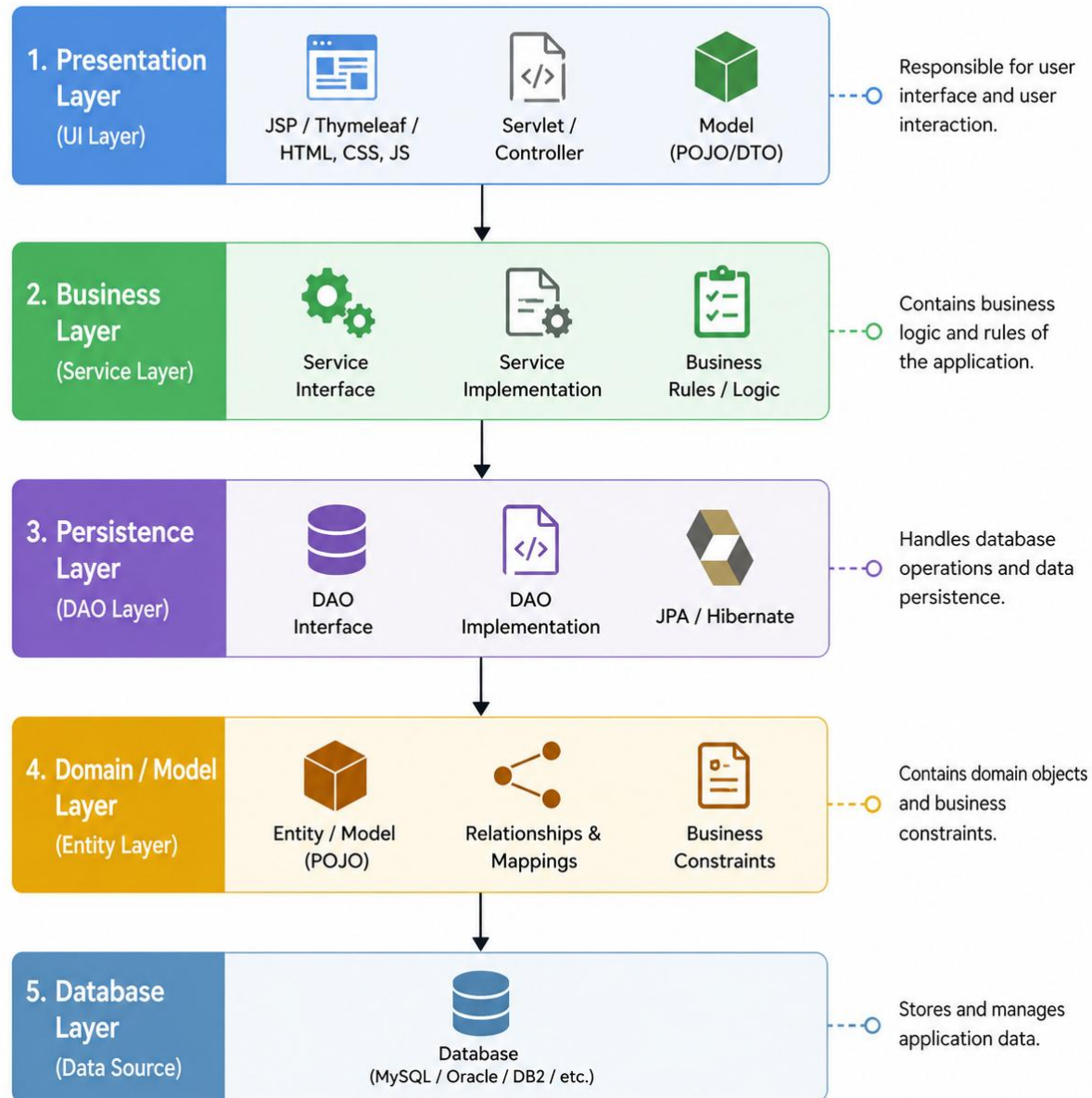
Keep general utilities in a util (or common) package. Don't scatter them across features.



## EXTRA TIPS

- Follow SOLID principles in package design.
- Refactor packages as the project evolves.
- Use IDE tools to enforce import and package conventions.
- Document package structure for new team members.

# Layered Architecture in Java





# LAYERED ARCHITECTURE — MINIMAL CODE SKELETON

*Each layer only calls the layer directly below it — Controller depends on Service, Service depends on Repository.*

```
// Repository layer -- data access only
public interface CustomerRepository {
    Customer findById(Long id);
}
// Service layer -- depends on Repository, not Controller
public class CustomerService {
    private final CustomerRepository repository;
    public CustomerService(CustomerRepository repository) {
        this.repository = repository;
    }
    public Customer getCustomer(Long id) {
        return repository.findById(id);
    }
}
// Controller layer -- depends on Service only
public class CustomerController {
    private final CustomerService service;
}
```

**KEY TAKEAWAY** Dependencies point one way: Controller → Service → Repository. Repository never imports Controller or Service types.

# TRY IT YOURSELF — EXERCISE 2: PACKAGE ORGANIZATION

*Reinforces: package-by-layer vs. package-by-feature and fully qualified naming you just covered.*

| STEP        | WHAT TO DO                                                                                  |
|-------------|---------------------------------------------------------------------------------------------|
| Goal        | Map seven CRM types to packages and fully qualified class names                             |
| File        | package-plan.md (in examples/module-08-exercises/)                                          |
| Types       | Seven types: Controller, Service, Repository, Customer, Request, AppConfig, exception class |
| Rule        | Lowercase packages, reverse-domain root com.northstar.crm, folder path matches declaration  |
| Deliverable | Seven FQCNs correct, DTO path matches declaration, packages lowercase and meaningful        |
| Reference   | exercises/exercise-02-package-plan.md                                                       |

```
$ cat package-plan.md
com.northstar.crm.controller.CustomerController
com.northstar.crm.service.CustomerService
com.northstar.crm.dto.CustomerRequest // src/main/java/com/northstar/crm/dto/...
com.northstar.crm.config.AppConfig // reverse-domain root: com.northstar.crm
```

**TRY IT NOW** Complete Exercise 2 before continuing — see exercises/exercise-02-package-plan.md.

# CROSS-CUTTING CONCERNS

*Functionalities that affect multiple layers or modules — applied consistently across the whole application.*



## Security

- Authentication, authorization, encryption, input validation.



## Logging

- Capturing application events for monitoring & troubleshooting.



## Monitoring & Metrics

- Health checks and performance monitoring.



## Exception Handling

- Centralized handling of exceptions and errors.



## Caching

- Improves performance by reusing frequently accessed data.



## Auditing

- Tracking user actions and changes for compliance.



## Configuration Mgmt

- Managing application configuration across environments.

**KEY TAKEAWAY** Handle cross-cutting concerns centrally with AOP, filters/interceptors, or framework support (Spring, Jakarta EE).

# CROSS-CUTTING CONCERN EXAMPLE: REQUEST LOGGING

*A single Filter applies logging to every request, without touching controller or service code.*

```
public class RequestLoggingFilter implements Filter {

    private static final Logger log =
        Logger.getLogger(RequestLoggingFilter.class.getName());

    @Override
    public void doFilter(ServletRequest req, ServletResponse res,
                        FilterChain chain) throws IOException, ServletException {
        long start = System.currentTimeMillis();
        try {
            chain.doFilter(req, res);
        } finally {
            long elapsed = System.currentTimeMillis() - start;
            log.info("Request handled in " + elapsed + "ms");
        }
    }
}
```

**KEY TAKEAWAY** Cross-cutting logic (logging, security, auditing) belongs in one shared component — not copy-pasted into every layer.



# DTO PACKAGE STRUCTURE

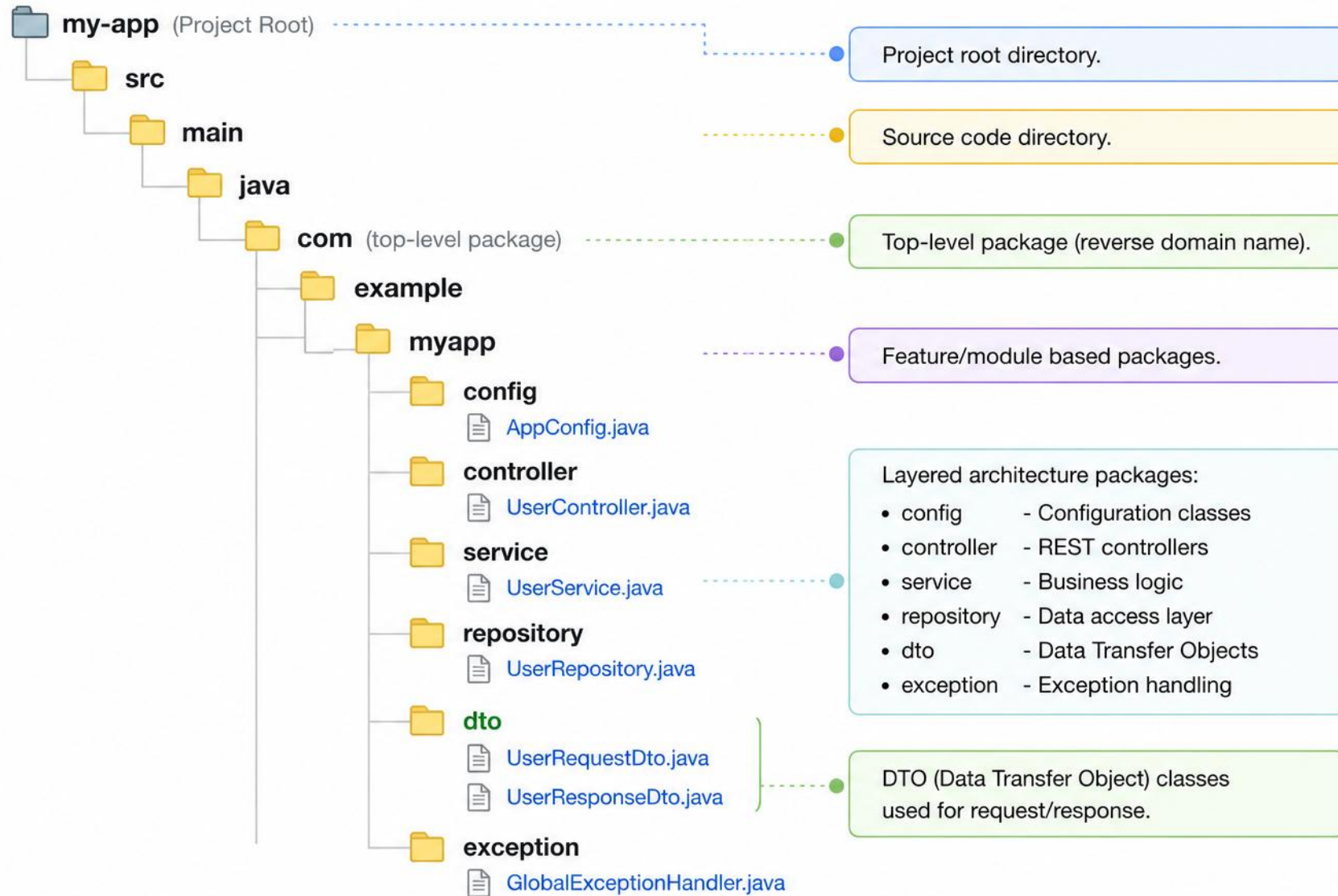
*DTOs (Data Transfer Objects) move data between layers or across application boundaries without exposing entities.*

```
com.example.project.dto/  
  request/                // CreateUserRequest.java, UpdateUserRequest.java  
  response/               // UserResponse.java, UserListResponse.java  
  common/                 // AddressDTO.java (shared across modules)  
  internal/               // UserSummaryDTO.java, UserSearchDTO.java  
  mapper/                 // UserMapper.java
```

| GUIDELINE                             | WHY                                      |
|---------------------------------------|------------------------------------------|
| Immutable, request/response separated | Do not expose internal entities directly |

**KEY TAKEAWAY** DTOs decouple layers, expose only required data, and improve performance and security.

# Java Package Structure Diagram



# HOW DTOS FLOW THROUGH THE APPLICATION

*Request DTOs enter at the Controller; Response DTOs leave the Service layer heading back out.*



## BEST PRACTICES

- Include only required fields; avoid business logic in DTOs; use meaningful and consistent naming.
- Validate request DTOs at the controller boundary using annotations (@NotNull, @Size, @Email).
- Use mapper frameworks (MapStruct, ModelMapper) or manual mapping for entity ↔ DTO conversion.

**KEY TAKEAWAY** A well-structured DTO package improves code organization, reusability, and maintainability.

# DTO CLASSES: REQUEST AND RESPONSE

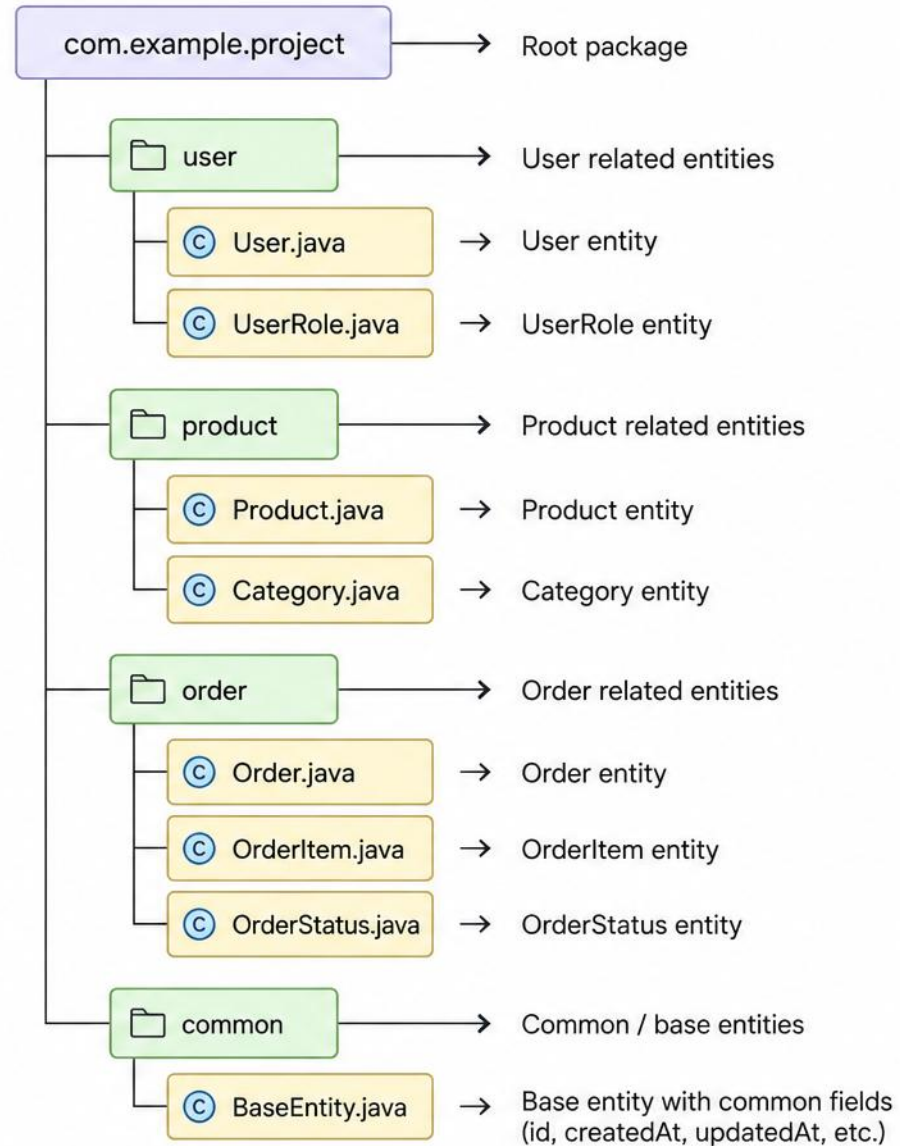
*Request DTOs validate input at the boundary; Response DTOs expose only what the client needs.*

```
public class CustomerRequest {
    @NotBlank(message = "Name is required")
    private String name;
    @Email(message = "Invalid email format")
    private String email;
    // getters and setters omitted for brevity
}

public class CustomerResponse {
    private String customerId;
    private String name;
    private String status;
    // getters and setters omitted for brevity
}
```

**KEY TAKEAWAY** CustomerRequest carries validated input in; CustomerResponse carries only safe, client-facing fields out.

# Java Entity Package Structure





# ENTITY DESIGN GUIDELINES

*Keep orchestration and application workflow in services. Entities may contain focused domain behavior that protects their own invariants, but should not contain infrastructure or controller logic.*



## Map to Tables

Each entity maps to a single database table.



## Use Relationships Properly

@OneToOne, @OneToMany,  
@ManyToMany.



## Surrogate Key

Prefer a surrogate key (Long id) over business keys.



## Auditing Fields

Use a base entity for createdBy,  
createdDate.



## Encapsulation

Keep fields private, use getters/setters.



## Protect Invariants

Focused domain behavior is fine; keep infrastructure/controller logic out.

## ANNOTATIONS TO KNOW (INDUSTRY PREVIEW)

- @Entity, @Table, @Id, @GeneratedValue, @Column, @OneToMany/@ManyToOne, @Version — implemented when JPA is introduced in a later module.

**KEY TAKEAWAY** INDUSTRY PREVIEW: lazy loading and @Version optimistic locking are covered when JPA is introduced in a later module.

# A JPA-STYLE ENTITY EXAMPLE

*Entities map to tables and hold their own fields. (Industry Preview — JPA annotations are implemented in a later module.)*

```
@Entity
@Table(name = "customers")
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @OneToMany(mappedBy = "customer")
    private List<Account> accounts;

    protected Customer() {
    }
}
```

**KEY TAKEAWAY** @Id + @GeneratedValue give every entity a surrogate key; relationships like @OneToMany model how tables connect.

# TRY IT YOURSELF — EXERCISE 3: ENTITY VS. DTO (STARTER)

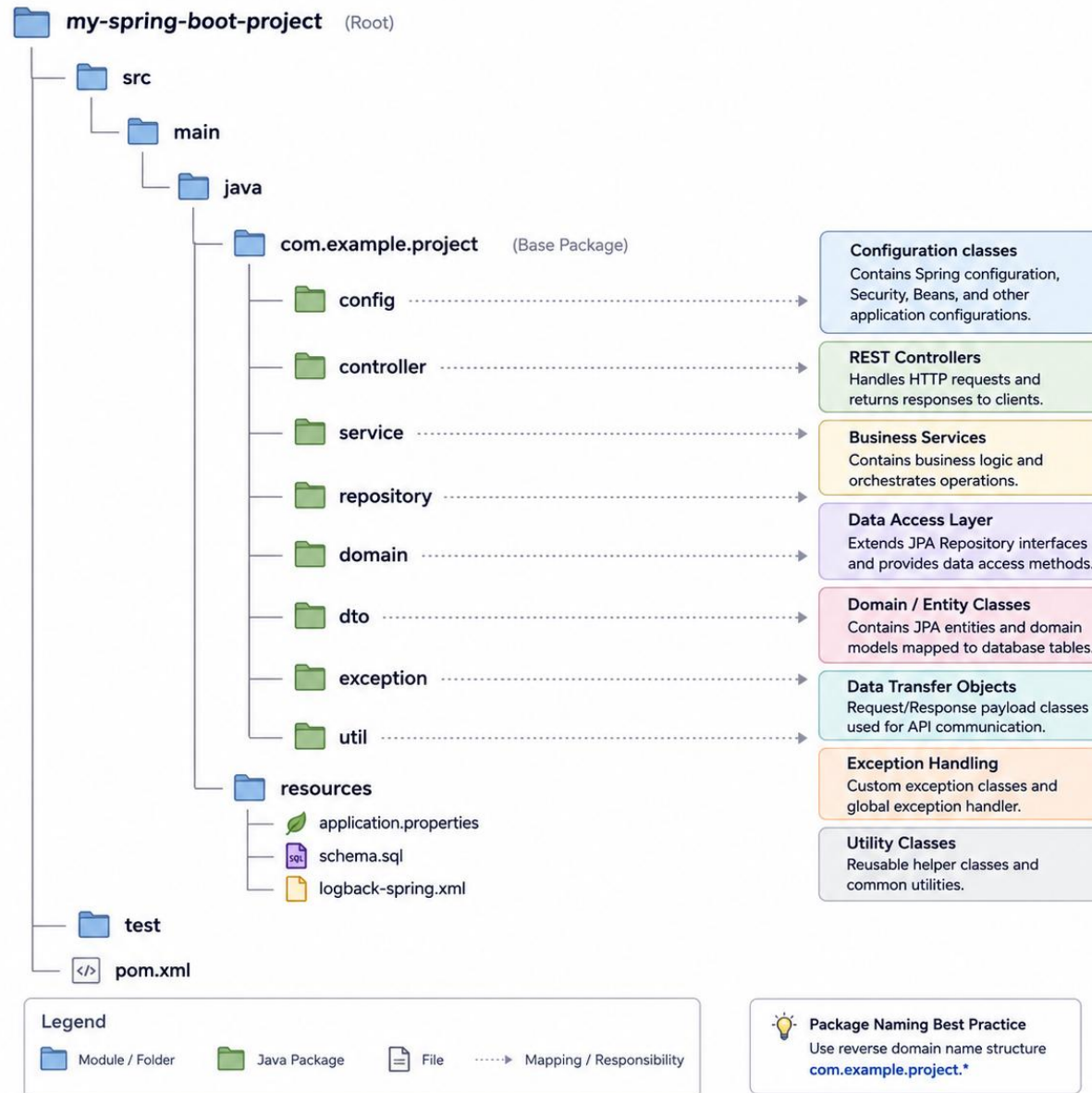
*Reinforces: entity vs. request/response DTO boundaries — a fill-in-the-blank starter, not from scratch.*

| STEP            | WHAT TO DO                                                                                   |
|-----------------|----------------------------------------------------------------------------------------------|
| Goal            | Prove a domain entity and boundary DTOs can have different fields and responsibilities       |
| Starter         | TODO / fill-in-the-blank skeleton — replace every ____ and // TODO; blanks do not compile    |
| Files           | mini-src/.../crm/: StructureDemo, entity/Customer, dto/CustomerRequest, dto/CustomerResponse |
| Entity vs. DTO  | Customer (id/name/status) vs. Request (name/email) vs. Response (+summary())                 |
| Expected output | CUS-1001   Amina Khan   ACTIVE                                                               |
| Reference       | exercises/exercise-03-entity-vs-dto.md                                                       |

```
$ javac -d mini-out mini-src/com/northstar/crm/entity/Customer.java \
    mini-src/com/northstar/crm/dto/Customer*.java \
    mini-src/com/northstar/crm/StructureDemo.java
$ java -cp mini-out com.northstar.crm.StructureDemo
// prints: CUS-1001 | Amina Khan | ACTIVE
```

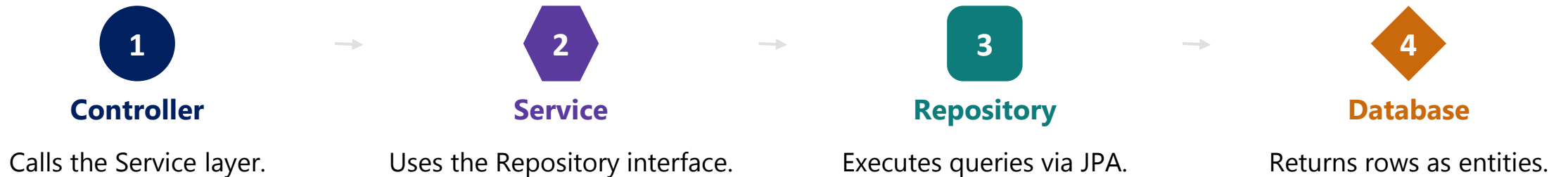
**TRY IT NOW** Complete Exercise 3 (fill in every blank, then compile) — see exercises/exercise-03-entity-vs-dto.md.

## Java Repository – Package Structure Diagram



# REPOSITORY — HOW IT FITS & BEST PRACTICES

*Prefer interfaces and let Spring Data JPA provide the implementation. (INDUSTRY PREVIEW — Spring Data JPA implementation starts in a later module.)*



## DESIGN GUIDELINES

- Use Interfaces — let Spring Data JPA provide implementations.
- Single Responsibility — one repository per aggregate/entity.
- Custom Queries — derived methods or @Query; keep logic minimal.
- Package Layout — group by feature: repository/user/, repository/product/, ...
- Custom Implementation — separate impl package, \*Impl suffix.
- Pagination & Sorting — use Pageable and Sort for efficiency.
- Specifications — use for dynamic, reusable queries.

## BEST PRACTICES

- Name repositories as <Entity>Repository; extend JpaRepository or CrudRepository.
- Keep methods intention-revealing and focused; avoid business logic in repositories.
- Repositories participate in database operations, but application transaction boundaries are generally defined in the service layer (@Transactional); write unit tests for custom implementations.

**KEY TAKEAWAY** A well-structured repository package ensures clean separation of data access logic.



# REPOSITORY INTERFACE EXAMPLE

*Spring Data JPA implements this interface at runtime — no implementation class to write. (Industry Preview.)*

```
public interface CustomerRepository
    extends JpaRepository<Customer, Long> {

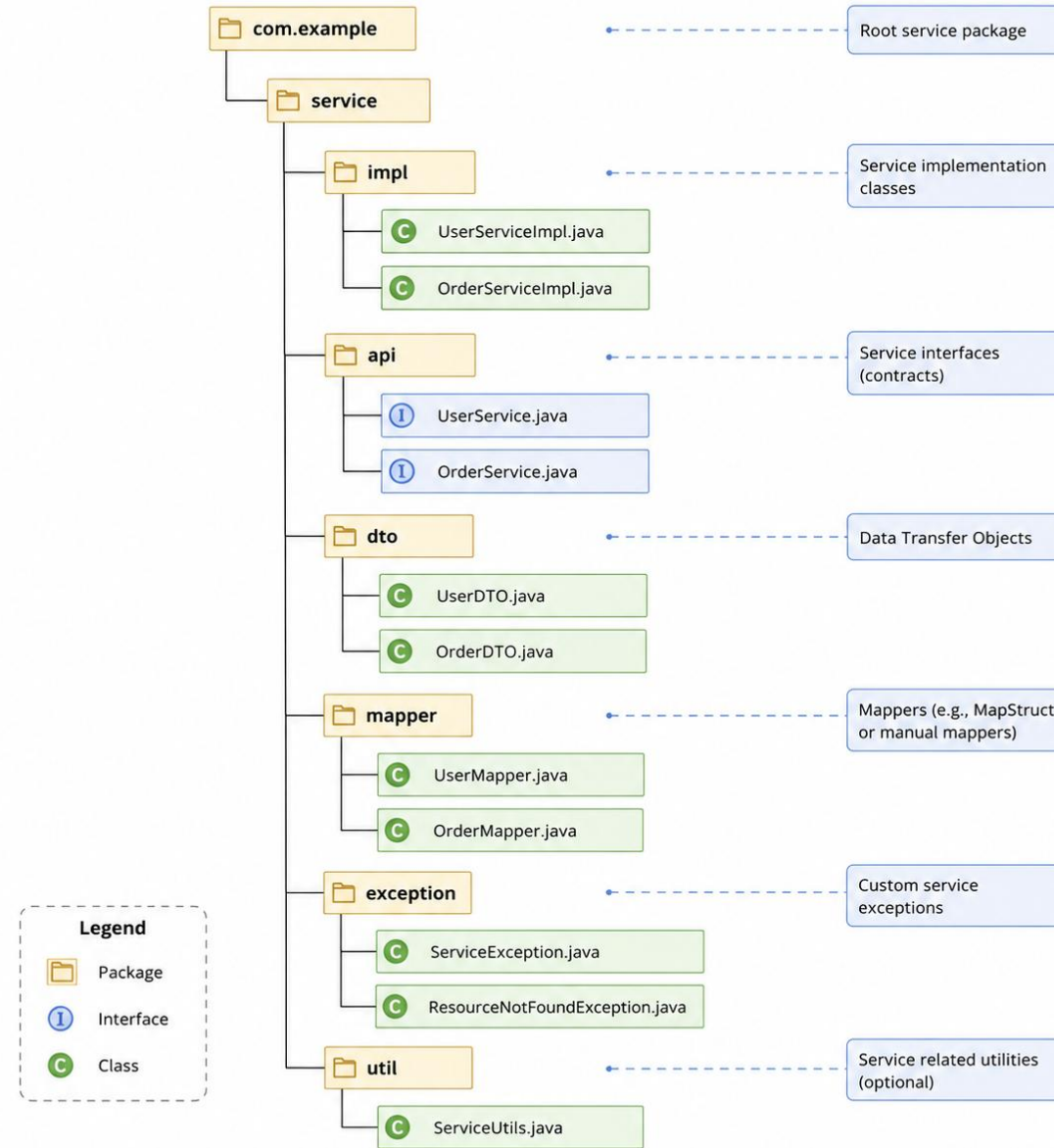
    Optional<Customer> findByEmail(String email);

    List<Customer> findByStatus(String status);

    @Query("SELECT c FROM Customer c WHERE c.name LIKE %:name%")
    List<Customer> searchByName(@Param("name") String name);
}
```

**KEY TAKEAWAY** Extending JpaRepository gives CRUD for free; derived methods and @Query cover everything else.

## Java Service Package Structure



# SERVICE DESIGN GUIDELINES

*Each service should focus on a single business concern. Avoid circular or convenience-only service dependencies — use service-to-service collaboration for legitimate orchestration.*



## Single Responsibility

One service, one business concern.



## Keep Stateless

Services should not hold request state.



## Validate at Service Layer

Business validation before repository calls.



## Use DTOs

Accept and return DTOs, not entities.



## Handle Exceptions

Translate exceptions into business exceptions.



## @Transactional

Manage transactions at the service layer. (Industry Preview.)

## MORE BEST PRACTICES

- Name services by business domain (e.g., UserService); follow the package layout service/<feature>/ with <Entity>Service interface plus <Entity>ServiceImpl implementation.
- Log important business events and follow a consistent code style; write unit tests for every service.
- Use service-to-service collaboration when it represents legitimate business orchestration, and consider a dedicated orchestration service for complex workflows.

**KEY TAKEAWAY** A well-structured service package improves maintainability and makes the application easier to test and scale.

# SERVICE INTERFACE AND IMPLEMENTATION

*The interface is the contract; the Impl class holds business logic and orchestrates the Repository.*

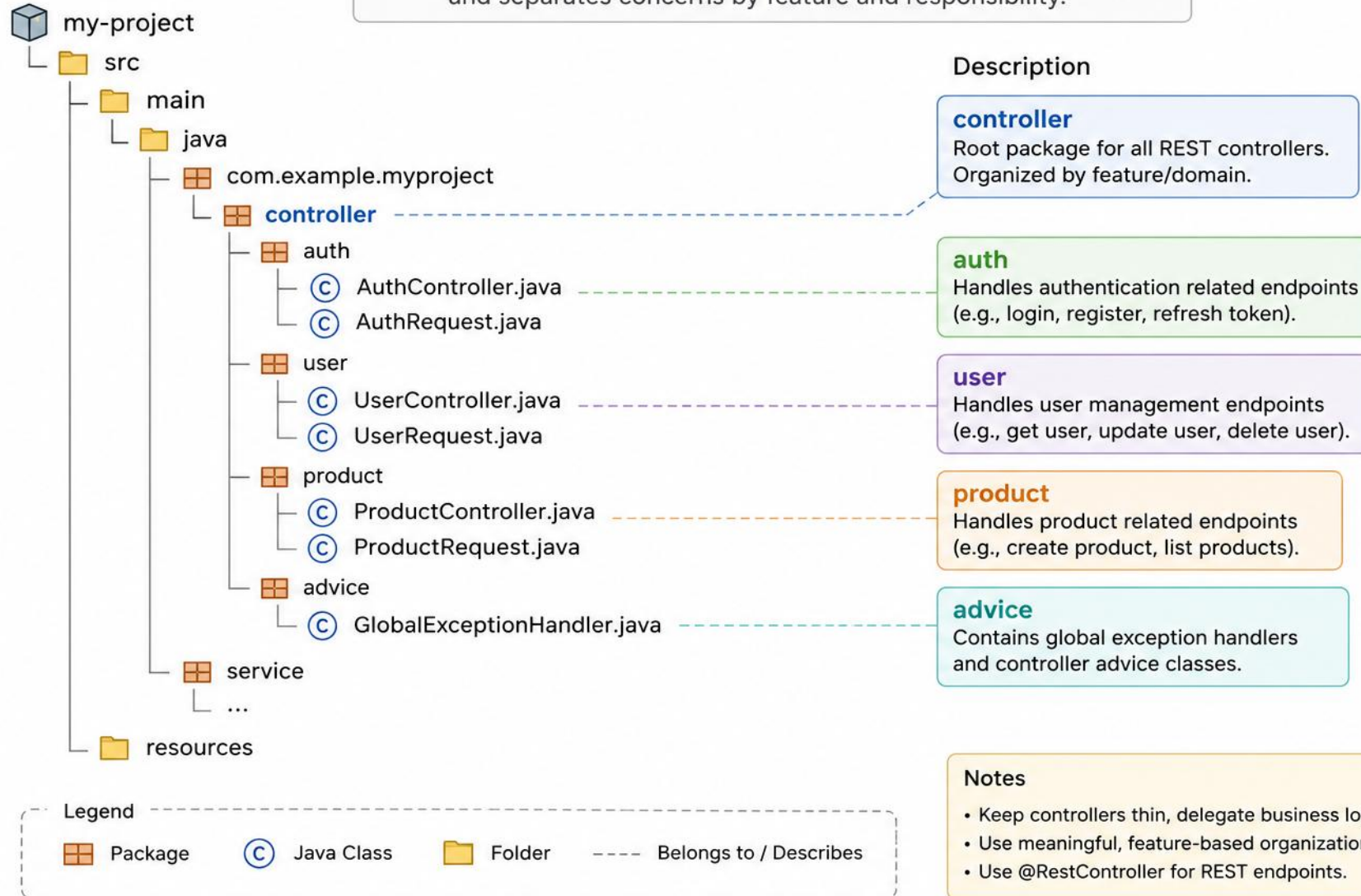
```
public interface CustomerService {
    CustomerResponse createCustomer(CustomerRequest request);
}

@Service
public class CustomerServiceImpl implements CustomerService {
    private final CustomerRepository repository;
    public CustomerServiceImpl(CustomerRepository repository) {
        this.repository = repository;
    }
    @Override
    @Transactional
    public CustomerResponse createCustomer(CustomerRequest request) {
        Customer saved = repository.save(new Customer(request.getName()));
        return new CustomerResponse(saved.getId(), saved.getName());
    }
}
```

**KEY TAKEAWAY** Controllers depend on the CustomerService interface, never on CustomerServiceImpl — the implementation can change freely.

# Java Controller Package Structure

This structure follows a layered architecture (e.g., Spring Boot) and separates concerns by feature and responsibility.





# CONTROLLER DESIGN GUIDELINES

*Keep controllers thin — delegate all business logic to the service layer. (Spring-specific items below are INDUSTRY PREVIEW, implemented in later modules.)*



## Thin Controllers

Delegate business logic to services.



## Request Validation

Use @Valid, @RequestParam annotations.



## Meaningful Mappings

Consistent and RESTful endpoint mappings.



## Use DTOs

Never return entities directly.



## Secure Endpoints

Use Spring Security (@PreAuthorize).



## Exception Handling

Use @ControllerAdvice globally.

## MORE BEST PRACTICES

- Return consistent API responses with proper HTTP status codes; write slice tests (@WebMvcTest).
- Use appropriate HTTP methods, pagination/sorting/filtering for list APIs, and document with OpenAPI/Swagger.
- Follow the package layout controller/<feature>/ with <Entity>Controller RESTful endpoint mappings under /api/<resource>.
- Broader UI concerns (web pages/views, static resources) live outside this backend package and are handled by front-end tooling.

**KEY TAKEAWAY** A well-structured controller package ensures clean API design and improved maintainability.

# REST CONTROLLER EXAMPLE

*Controllers stay thin — validate input, delegate to the Service, and shape the HTTP response. (Industry Preview.)*

```
@RestController
@RequestMapping("/api/customers")
public class CustomerController {
    private final CustomerService service;

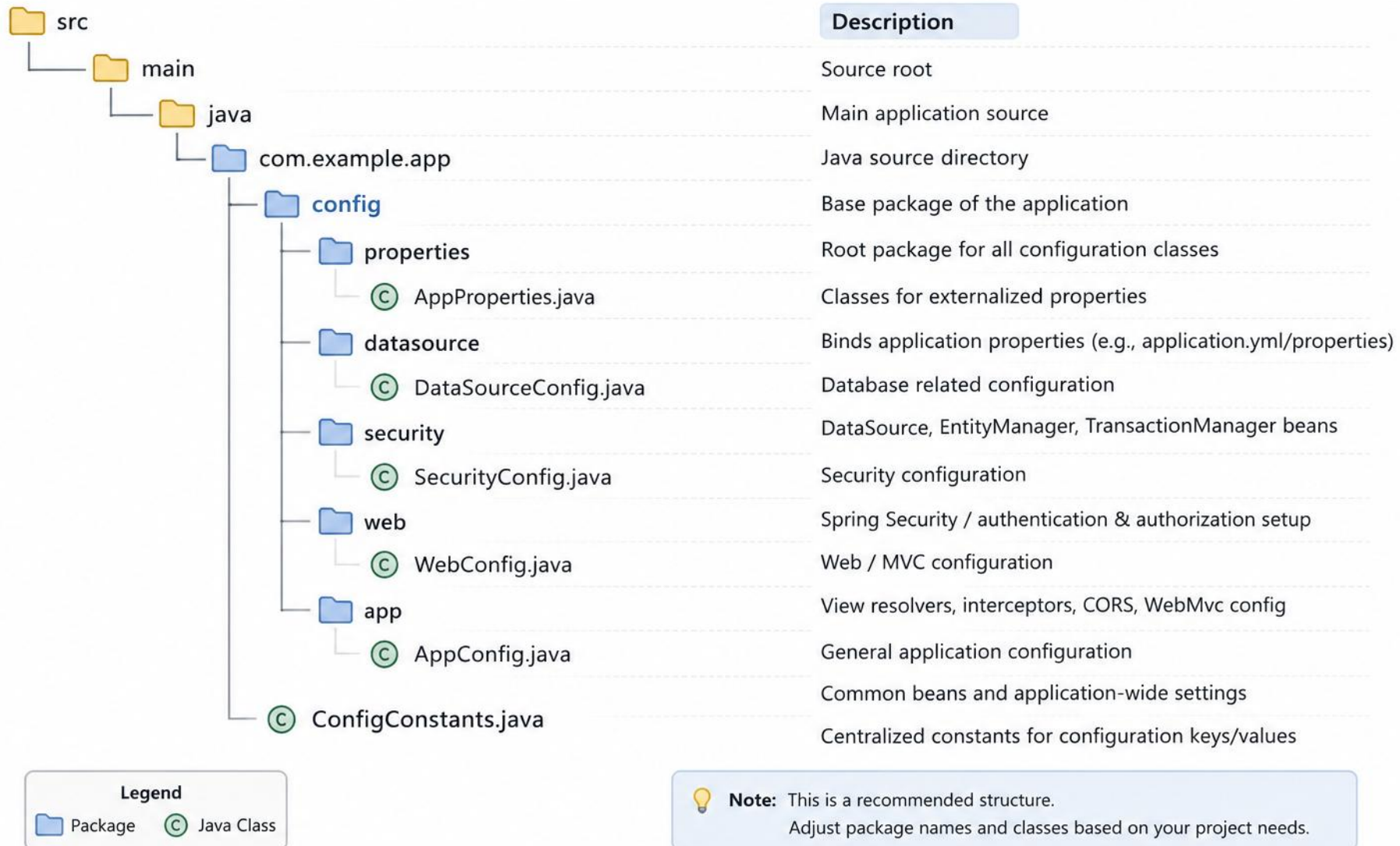
    public CustomerController(CustomerService service) {
        this.service = service;
    }

    @PostMapping
    public ResponseEntity<CustomerResponse> create(
        @Valid @RequestBody CustomerRequest request) {
        return ResponseEntity.ok(service.createCustomer(request));
    }
}
```

**KEY TAKEAWAY** @Valid triggers the DTO's validation annotations; the controller never touches Customer, only DTOs.

# Java Configuration Package Structure

Typical package structure for managing application configuration in Java



# CONFIGURATION DESIGN GUIDELINES

*Keep configuration classes clean and externalized — never mix business logic into config. (Spring-specific items below are INDUSTRY PREVIEW, implemented in later modules.)*



## Modular Configuration

Group config by concern: security, datasource, web.



## Externalize Properties

Use application.yml, not hardcoded values.



## Profile-Based Setup

Use @Profile for dev, test, and prod.



## @Configuration & @Bean

Define and wire beans explicitly.



## Enable Validation

Validate properties with @Validated.



## Keep It Clean

No business logic — config only wires components.

## MORE BEST PRACTICES

- Keep all configuration classes in a dedicated config package; use meaningful class and method names.
- Secure sensitive properties with environment variables or a secrets manager; document configs and write unit tests.

**KEY TAKEAWAY** A well-structured configuration package centralizes settings, promotes consistency, and makes the application easier to manage, test, and deploy across environments.

# CONFIGURATION CLASS EXAMPLE

*@Configuration classes wire beans explicitly — no business logic belongs here. (Industry Preview.)*

```
@Configuration
public class AppConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    @ConfigurationProperties(prefix = "app.datasource")
    public DataSourceProperties dataSourceProperties() {
        return new DataSourceProperties();
    }
}
```

**KEY TAKEAWAY** @Bean methods build objects the framework should manage; @ConfigurationProperties binds externalized settings to a typed class.

# EXCEPTION PACKAGE STRUCTURE

*Custom exceptions used across the application, handled consistently through a global handler.*

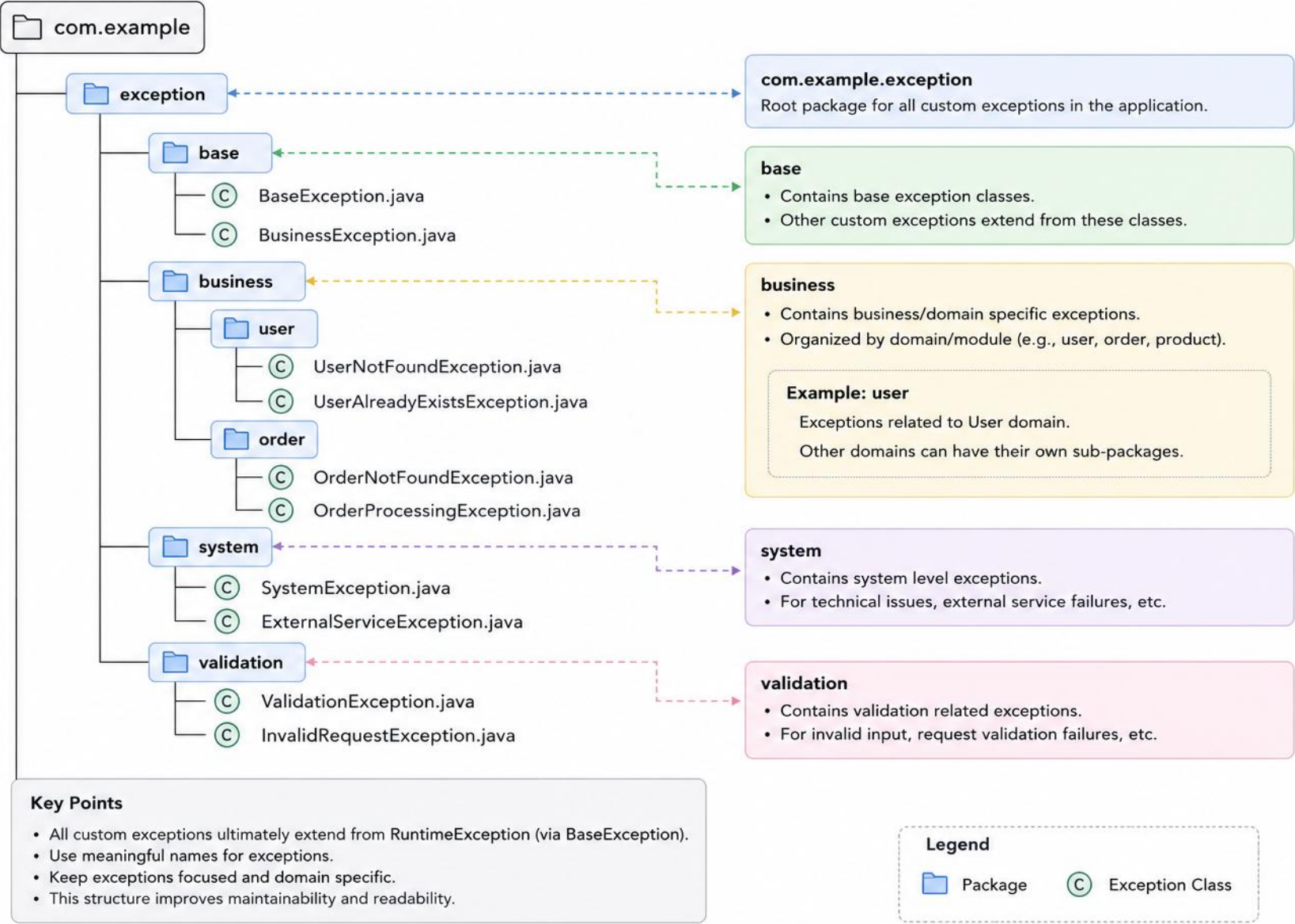
```
com.example.project.exception/  
  base/                // BaseException.java, BusinessException.java  
  business/            // ResourceNotFoundException.java, DuplicateResourceException.java  
  handler/             // GlobalExceptionHandler.java, ErrorResponse.java  
  validation/          // ConstraintViolationException.java  
  security/            // AuthenticationException.java, AuthorizationException.java
```

- Apply the Module 7 exception-handling standards (base hierarchy, specific exception types, never leak internals to clients) within this package structure.

**KEY TAKEAWAY** A well-structured exception package improves error handling — see Module 7 for detailed exception-handling standards.



# Java Exception Package Structure



# CUSTOM EXCEPTION AND GLOBAL HANDLER

*A focused exception type paired with a centralized handler keeps error handling consistent everywhere.*

```
public class CustomerNotFoundException extends RuntimeException {
    public CustomerNotFoundException(Long id) {
        super("Customer not found: " + id);
    }
}

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(CustomerNotFoundException.class)
    public ResponseEntity<ErrorResponse> handle(
        CustomerNotFoundException ex) {
        ErrorResponse error = new ErrorResponse(ex.getMessage());
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }
}
```

**KEY TAKEAWAY** Business code throws specific exceptions; one `@RestControllerAdvice` class turns every one of them into a clean HTTP response.

# TRY IT YOURSELF — EXERCISE 4: LAYER RESPONSIBILITIES

*Reinforces: the controller / service / repository / entity / dto / config / exception boundaries just covered.*

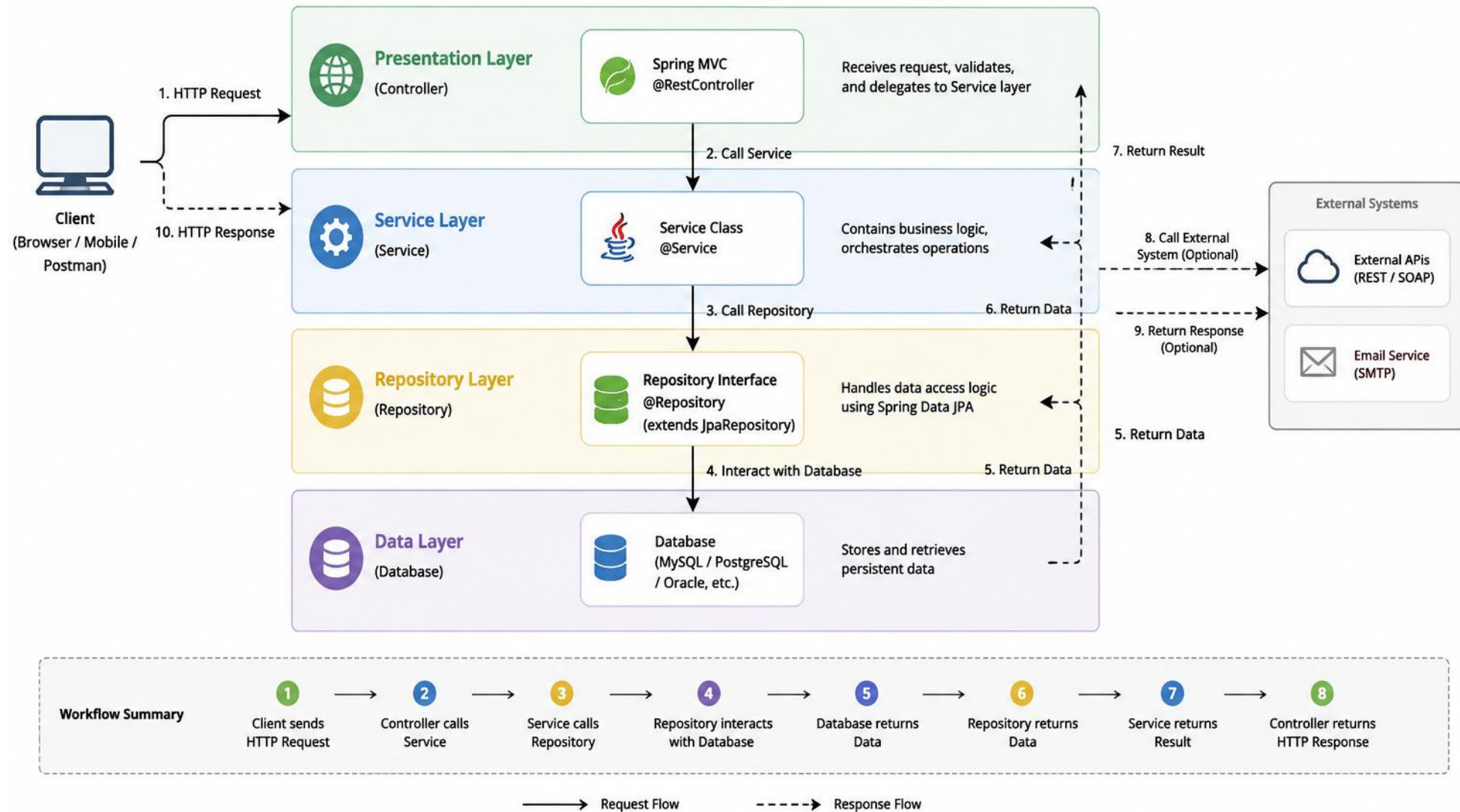
| STEP        | WHAT TO DO                                                                                 |
|-------------|--------------------------------------------------------------------------------------------|
| Goal        | Assign each CRM task to the layer that should own it                                       |
| File        | layer-responsibilities.md (in examples/module-08-exercises/)                               |
| Assign      | Seven tasks: input handling, validation, find-by-ID, state, request shape, failure, wiring |
| Repair      | Rewrite a "god controller" that validates rules, edits a list, and builds queries itself   |
| Deliverable | Seven tasks assigned correctly, god-controller flow repaired, 2+ benefits explained        |
| Reference   | exercises/exercise-04-layer-responsibilities.md                                            |

```
$ cat layer-responsibilities.md
accept input -> controller      validate -> service
find by ID -> repository        domain state -> entity
request shape -> dto            failure -> exception, wiring -> config
// repaired: Controller -> Service -> Repository -> Service -> Controller
```

**TRY IT NOW** Complete Exercise 4 before continuing — see exercises/exercise-04-layer-responsibilities.md.

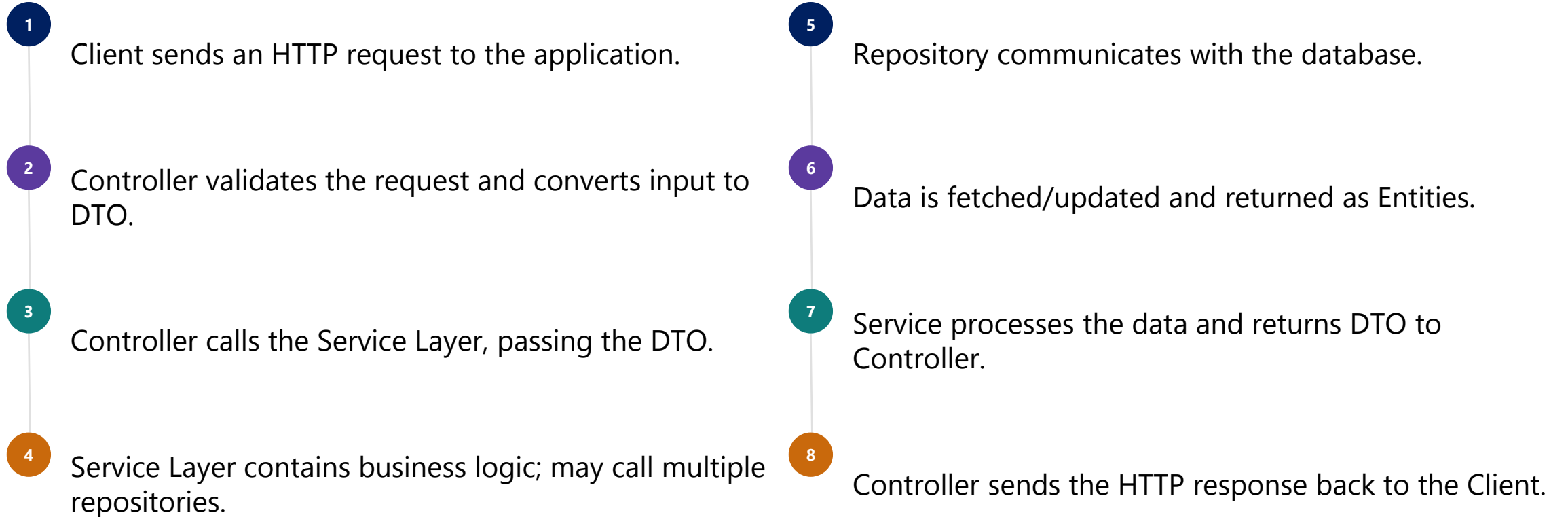
# Layered Architecture Workflow (Java)

Request Flow from Client to Database and Back



# DETAILED REQUEST FLOW

*Eight steps take a request from the client, through every layer, and back again.*



**KEY TAKEAWAY** Each layer can be tested in isolation using mocks — that's the payoff of a clean, layered request flow.

# REQUEST FLOW, TRACED THROUGH REAL CODE

*The same eight steps, expressed as an actual Controller -> Service -> Repository call chain.*

```
@PostMapping                                // 1-2. Client -> Controller
public ResponseEntity<CustomerResponse> create(
    @Valid @RequestBody CustomerRequest request) {
    return ResponseEntity.ok(
        service.createCustomer(request));    // 3. Controller -> Service
    }

public CustomerResponse createCustomer(CustomerRequest request) {
    Customer entity = new Customer(request.getName());
    Customer saved = repository.save(entity); // 4-6. Service -> Repo -> DB
    return new CustomerResponse(
        saved.getId(), saved.getName());    // 7-8. Service -> Controller
    }
```

**KEY TAKEAWAY** Every arrow in the request-flow diagram is one method call in real code — nothing magic happens between the layers.



# TRY IT YOURSELF — EXERCISE 5: TRACE A CUSTOMER REQUEST

*Reinforces: the controller → service → repository request flow you just saw, success and failure.*

| STEP         | WHAT TO DO                                                                               |
|--------------|------------------------------------------------------------------------------------------|
| Goal         | Document how a future create-customer request for CUS-1001 moves through the layers      |
| File         | customer-request-flow.md (in examples/module-08-exercises/)                              |
| Success flow | Client -> Controller -> Service (validate, assign ID/status) -> Repository -> Controller |
| Failure flow | A blank name stops at the Service layer — Repository is never called                     |
| Deliverable  | Success + failure diagrams, DTO/entity transformations, a truthful now-vs-later table    |
| Reference    | exercises/exercise-05-request-flow.md                                                    |

```
$ cat customer-request-flow.md
Name: Amina Khan   Email: amina@example.test
Requested status: ACTIVE   Correlation ID: lab-request-001
  // now: package names, stub responsibilities, documented flow
  // later: Spring annotations, JPA repository, HTTP response mapping
```

**TRY IT NOW** Complete Exercise 5 before continuing — see exercises/exercise-05-request-flow.md.

# DEPENDENCY RULES AND BEST PRACTICES

*Controller → Service → Repository: dependencies point one way only — lower layers never depend on upper ones.*

## KEY BENEFITS



### Separation of Concerns

Each layer has one responsibility.



### Maintainability

Changes in one layer stay local.



### Testability

Each layer tested in isolation with mocks.



### Flexibility

Tech changes in one layer don't affect others.



### Reusability

Services and components reused across modules.

## BEST PRACTICES

- Keep controllers thin — delegate all logic to services; use DTOs for data transfer between layers.
- Do not expose entities outside the service layer; use meaningful names for packages and classes.
- Handle exceptions and logging consistently across layers; write unit tests for each layer.
- Dependency direction (the rule, stated once): Controller → Service → Repository — lower layers (e.g., Repository) must never depend on controllers or web/HTTP concerns.
- Modularity: each layer is independently replaceable.

**KEY TAKEAWAY** Dependencies always flow Controller → Service → Repository — this keeps the architecture modular, testable, and maintainable.

# TRY IT YOURSELF — EXERCISE 6: DEPENDENCY DIRECTION

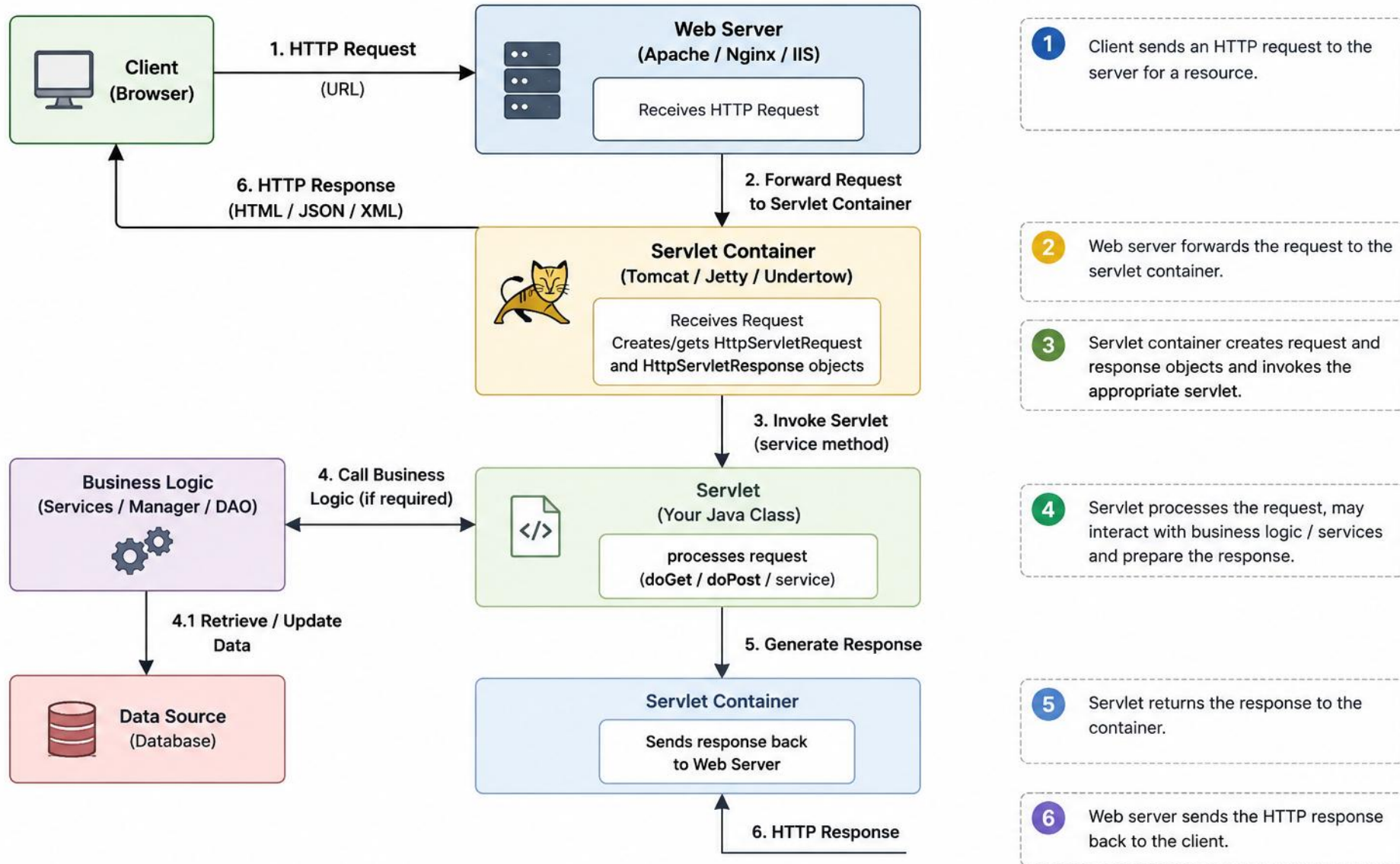
*Reinforces: the Controller → Service → Repository dependency rule you just covered.*

| STEP        | WHAT TO DO                                                                                 |
|-------------|--------------------------------------------------------------------------------------------|
| Goal        | Identify acceptable vs. problematic dependencies before they turn circular                 |
| File        | architecture-rules.md (in examples/module-08-exercises/)                                   |
| Classify    | Seven dependencies across controller, service, repository, entity, dto — acceptable or not |
| Cycle       | controller->service->repository->controller is a cycle; repair ends at ->entity            |
| Deliverable | Seven dependencies classified, the cycle repaired, one written architecture rule           |
| Reference   | exercises/exercise-06-dependency-direction.md                                              |

```
$ cat architecture-rules.md
entity -> controller      // PROBLEMATIC: domain depends on transport
repository -> controller  // PROBLEMATIC: persistence depends on UI
service -> DTO            // needs context – avoid transport leakage
// rule: entity/repository packages must never import controller classes
```

**TRY IT NOW** Complete Exercise 6 before continuing — see exercises/exercise-06-dependency-direction.md.

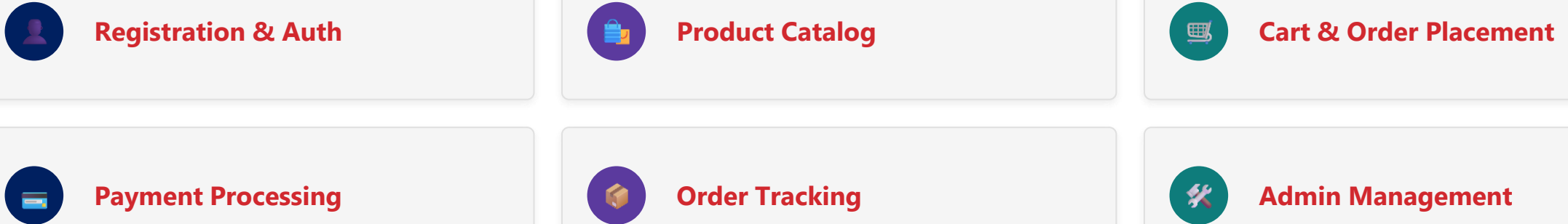
# Java Request Processing Flow



# ENTERPRISE PROJECT EXAMPLE

*Online Retail Order Management System — customers browse products, place orders, and track status.*

## KEY FEATURES



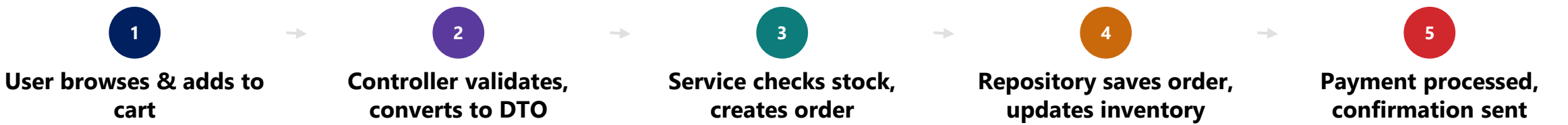
## TECHNOLOGY STACK



**KEY TAKEAWAY** Domain: E-Commerce/Retail — build a scalable, secure, and maintainable Order Management System. A layered, well-organized backend also improves the customer experience — faster responses, fewer errors.

# EXAMPLE FLOW: PLACE ORDER

*One customer action, five layers of coordinated work.*



| PACKAGE                            | RESPONSIBILITY         | PACKAGE                           | RESPONSIBILITY            |
|------------------------------------|------------------------|-----------------------------------|---------------------------|
| <code>com.retail.controller</code> | Handles HTTP requests  | <code>com.retail.dto</code>       | Data Transfer Objects     |
| <code>com.retail.service</code>    | Business logic & rules | <code>com.retail.exception</code> | Custom exceptions         |
| <code>com.retail.repository</code> | Data access layer      | <code>com.retail.config</code>    | Configuration classes     |
| <code>com.retail.entity</code>     | JPA entities / models  | <code>com.retail.util</code>      | Utility classes & helpers |

**KEY TAKEAWAY** This enterprise project shows how layered architecture delivers a robust, scalable, maintainable solution.



# KEY MODULES & SAMPLE ENTITIES

*The Order Management System is organized into five business-facing modules, backed by seven core entities.*

## KEY MODULES

**Product Mgmt**  
Products, categories, inventory.

**Order Mgmt**  
Create, update, cancel, track orders.

**Payment Mgmt**  
Process payments and refunds.

**User Mgmt**  
Customers, roles, permissions.

**Reporting**  
Sales and order reports.

## SAMPLE ENTITIES

| ENTITY   | DESCRIPTION              | ENTITY    | DESCRIPTION                 |
|----------|--------------------------|-----------|-----------------------------|
| User     | Stores user information  | Payment   | Payment transaction details |
| Product  | Stores product details   | Inventory | Tracks stock availability   |
| Category | Product categories       | OrderItem | Items in an order           |
| Order    | Order header information |           |                             |

# LAB OVERVIEW — PROJECT STRUCTURE AND ORGANIZATION

*Build the Northstar CRM Maven skeleton — plain Java packages and stub classes (no Spring Boot yet). Timed path ~45 min in class; full path 3–4 hrs for homework.*

## LAB OBJECTIVES

- 1 Create the standard Maven layout and seven layer packages.
- 2 Separate DTO (API contracts) from entity (domain model).
- 3 Exercises 1–6 (already completed throughout this module) prepared you for this — now trace CUS-1001 through the layers on paper.

## LAB CONSTRAINTS

- Plain Java 21 + Maven only — no Spring Boot, JPA, or HTTP framework.
- Seven layer packages with stub classes: controller, service, repository, entity, dto, config, exception.
- Stub methods throw UnsupportedOperationException by design — proves compile-time correctness before behavior.
- Target architecture: apply the layered-architecture diagram and dependency rules covered earlier in this module.

**KEY TAKEAWAY** A compiled, documented, stub-only skeleton — no Spring, JPA, or HTTP required yet.

# LAB TASKS AND DELIVERABLES

*Build the plain-Java Northstar CRM skeleton, complete the tasks, and submit the required deliverables.*

| # | TASK                       | DESCRIPTION                                                                                                     |
|---|----------------------------|-----------------------------------------------------------------------------------------------------------------|
| 1 | Project Setup              | Create lab8-crm; add pom.xml (JDK 21, maven-compiler-plugin) and .gitignore                                     |
| 2 | Package Structure          | Create controller, service, repository, entity, dto, config, exception (seven packages under com.northstar.crm) |
| 3 | Entity & DTO Stubs         | Customer entity plus CustomerRequest/CustomerResponse DTOs — empty shells, no JPA                               |
| 4 | Repository Layer           | CustomerRepository.findById()/save() throw UnsupportedOperationException on purpose                             |
| 5 | Service Layer              | CustomerService constructor-injects the repository; create()/getById() throw for now                            |
| 6 | Controller & Main          | CustomerController delegates to service; Main prints banner; mvn clean compile + run                            |
| 7 | Config & Exception Stubs   | AppConfig placeholder; CustomerNotFoundException(customerId)                                                    |
| 8 | Docs & Failure Experiments | docs/layer-flow.md, docs/CODING-STANDARDS.md; run 3+ documented failure experiments                             |

**KEY TAKEAWAY** These tasks prove a compile-ready, correctly layered skeleton — ready for Lab 9.

# DELIVERABLES & EVALUATION CRITERIA

*What to submit, and how the lab is scored.*

## DELIVERABLES

 **Source Code**

 **Docs & Standards**

 **Compile Evidence**

 **Failure Evidence**

 **README File**

- Maven skeleton (7 packages + stubs + Main) · CODING-STANDARDS.md + layer-flow.md · BUILD SUCCESS compile evidence · 3+ failure experiments · project README.

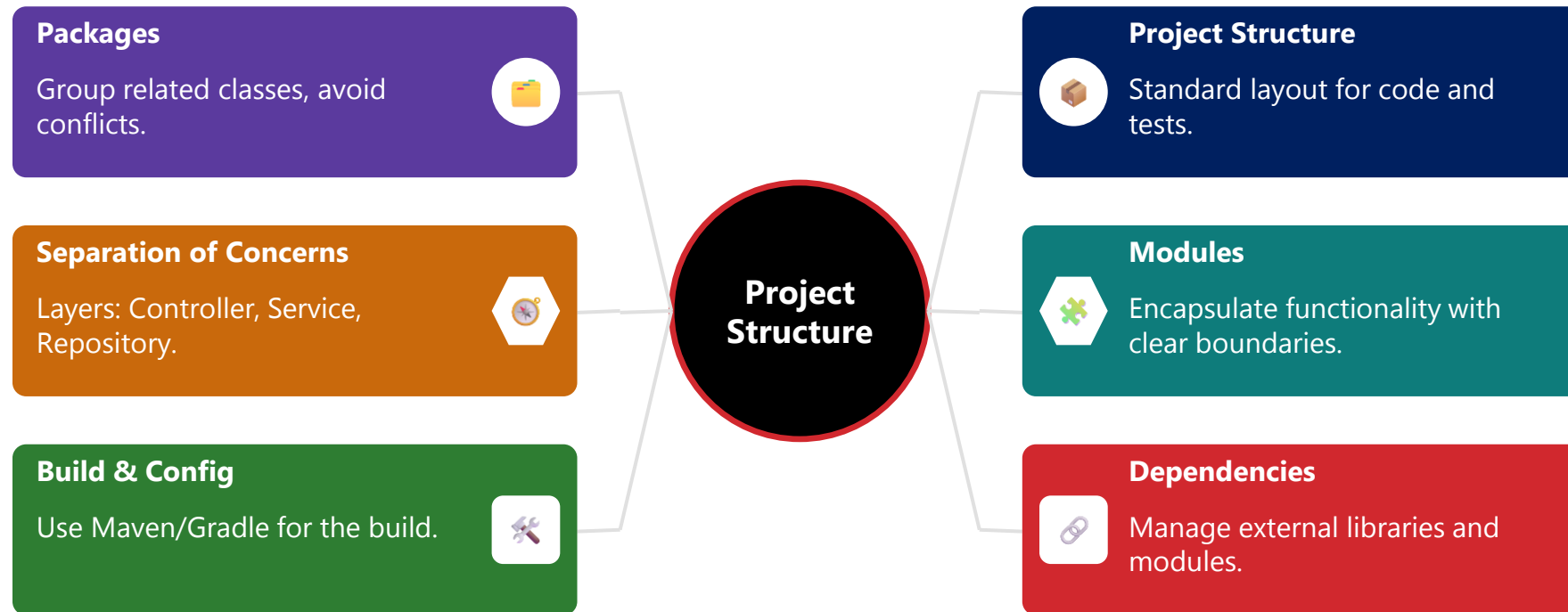
| EVALUATION CRITERIA                                      | WEIGHT |
|----------------------------------------------------------|--------|
| Core Implementation (packages, stubs, standards)         | 30%    |
| Environment, Structure & Compile Correctness             | 25%    |
| Failure Handling (stub failures / layer-rule experiment) | 15%    |
| Automated Verification (mvn compile, Main run)           | 10%    |
| Security & Production Awareness                          | 10%    |
| Documentation & Evidence                                 | 10%    |

**KEY TAKEAWAY** Keep layers independent, prove compile success, and document decisions early.

# MODULE 8 SUMMARY AND MODULARIZATION CHECKLIST

*Organize Java applications using a well-defined project structure and modular design.*

**MODULARIZATION CHECKLIST** High cohesion, low coupling, and encapsulation • Reusability and scalability • Standard layout (src/main/java, src/test/java) • Clear, reverse-domain package naming • Refactor structure regularly • Avoid cyclic dependencies • Faster onboarding and easier debugging.



**KEY TAKEAWAY** A well-structured and modular Java project — built on high cohesion, low coupling, and consistent naming — is easier to maintain, extend, and scale, leading to reliable software.

# KNOWLEDGE CHECK (1 OF 3)

*Test your understanding of the concepts covered in this module.*

## 1. What is the primary benefit of layered architecture?

- A. It increases code duplication
- B. It tightly couples all components
- C. Separation of concerns and maintainability
- D. It makes the application slower

## 3. What is the purpose of DTOs?

- A. To define database tables
- B. To transfer data between layers
- C. To handle exceptions
- D. To configure the application

## 2. Which layer handles HTTP requests and responses?

- A. Service Layer
- B. Repository Layer
- C. Controller Layer
- D. Entity Layer

## 4. (Industry Preview) Which annotation handles exceptions globally in Spring Boot?

- A. @Catch
- B. @ExceptionHandler
- C. @ControllerAdvice
- D. @ExceptionHandler

**KEY TAKEAWAY** Understanding each layer's role and proper package organization helps build robust, maintainable applications.



# KNOWLEDGE CHECK (2 OF 3)

*Four more questions to check your understanding.*

**5. (Industry Preview) Which Spring Data JPA interface provides CRUD operations and query methods?**

- A. CrudRepository
- B. JpaRepository**
- C. EntityManager
- D. JdbcTemplate

**7. (Industry Preview) Which annotation marks a class as a REST controller?**

- A. @Service
- B. @Component
- C. @RestController**
- D. @Repository

**6. (Industry Preview) What is the purpose of the @Valid annotation?**

- A. To validate database entries
- B. To validate incoming request data**
- C. To mark a class as a valid entity
- D. To validate application properties

**8. Which layer is directly responsible for interacting with the database?**

- A. Service Layer
- B. Controller Layer
- C. Repository Layer**
- D. DTO Layer

**KEY TAKEAWAY** Spring Data JPA's JpaRepository extends CrudRepository with paging, sorting, and batch operations.

# KNOWLEDGE CHECK (3 OF 3)

*Two final questions, plus the full answer key.*

**9. What is the recommended way to pass data from Service Layer to Controller Layer?**

- A. Entity
- B. DTO**
- C. Map
- D. List

**10. (Industry Preview) Which file is typically used to configure database connection and other app properties?**

- A. application.yml (or .properties)**
- B. web.xml
- C. logback.xml
- D. pom.xml

## ANSWER KEY

• **1-C 2-C 3-B 4-C 5-B 6-B 7-C 8-C 9-B 10-A**

**KEY TAKEAWAY** Understanding the roles of each layer, proper package organization, and best practices helps in building robust, scalable, and maintainable Java applications.

## MODULE 8 COMPLETE

# Java Project Structure and Modularization

You can now design and structure Java applications that are well-organized, modular, and easy to maintain and extend.